

ui.highchart 全面详细阐述

ui.highchart 是 NiceGUI 框架中集成 Highcharts 图表库的元素，用于在 NiceGUI 应用中快速创建交互式图表。由于 Highcharts 的许可限制，它未包含在 NiceGUI 标准包中，需单独安装后使用，支持多种图表类型、动态更新、事件回调及扩展依赖等功能，适用于数据可视化场景。

一、基础使用前提

1. 安装方式

因不属于 NiceGUI 标准组件，需通过 pip 命令安装扩展依赖：

```
pip install nicegui[highcharts]
```

2. 核心作用

基于 Highcharts 库实现图表渲染，支持 Highcharts 原生的大部分配置选项，同时适配 NiceGUI 的组件化特性，可与其他 NiceGUI 元素（如按钮、输入框）联动，实现图表的动态更新与交互。

二、初始化参数

初始化 `ui.highchart()` 时支持以下核心参数，用于定义图表的基础配置、类型、依赖及事件回调：

参数名	说明	示例
<code>options</code>	Highcharts 配置字典，包含图表类型、数据、坐标轴、标题等所有核心配置	<code>{'chart': {'type': 'bar'}, 'series': [{'data': [1,2]}]}</code>
<code>type</code>	图表类型，对应 Highcharts 实例类型（默认值为 "chart"）	"stockChart"（股票图）、"mapChart"（地图）
<code>extras</code>	额外依赖列表，用于启用 Highcharts 非默认模块（如仪表盘、可拖拽点）	<code>['solid-gauge', 'draggable-points']</code>
<code>on_point_click</code>	点点击事件回调函数，点击图表数据点时触发	<code>lambda e: ui.notify(f'点击数据: {e}')</code>
<code>on_point_drag_start</code>	点拖拽开始事件回调函数	<code>lambda e: ui.notify('拖拽开始')</code>
<code>on_point_drag</code>	点拖拽过程事件回调函数	-
<code>on_point_drop</code>	点拖拽结束（释放）事件回调函数	<code>lambda e: ui.notify('拖拽结束')</code>

三、核心功能与示例

1. 基础图表创建

通过 `options` 参数配置图表类型、数据、坐标轴等，支持 Highcharts 原生所有图表类型（如柱状图、折线图、饼图等）。

示例：柱状图

```
from nicegui import ui
from random import random

# 创建柱状图
chart = ui.highchart({
    'title': False, # 隐藏标题
    'chart': {'type': 'bar'}, # 图表类型为柱状图
    'xAxis': {'categories': ['A', 'B']}, # X轴分类
    'series': [ # 数据系列
        {'name': 'Alpha', 'data': [0.1, 0.2]},
        {'name': 'Beta', 'data': [0.3, 0.4]},
    ],
}).classes('w-full h-64') # 设置宽高样式

# 动态更新图表数据的按钮
def update():
    # 修改第一个系列第一个数据点为随机值
    chart.options['series'][0]['data'][0] = random()
    chart.update() # 刷新图表

ui.button('更新数据', on_click=update)
ui.run()
```

2. 启用额外依赖（扩展图表类型）

部分图表类型（如仪表盘、可拖拽点）需要 Highcharts 额外模块支持，通过 `extras` 参数指定依赖名称即可启用。

示例：仪表盘图表 (solid-gauge)

```
from nicegui import ui

ui.highchart({
    'title': False,
    'chart': {'type': 'solidgauge'}, # 仪表盘类型
    'yAxis': {
        'min': 0, # 最小值
        'max': 1, # 最大值
    },
    'series': [{'data': [0.42]}], # 当前值
}, extras=['solid-gauge']).classes('w-full h-64') # 启用仪表盘依赖
ui.run()
```

3. 可拖拽数据点（交互功能）

通过 `extras=['draggable-points']` 启用拖拽功能，并配置 `plotOptions.series.dragDrop` 定义拖拽规则，结合事件回调实现拖拽交互反馈。

示例：可拖拽点的图表

```
from nicegui import ui

ui.highchart(
    {
        'title': False,
        'plotOptions': {
            'series': {
                'stickyTracking': False,
                'dragDrop': {
                    'draggableY': True, # 允许Y轴方向拖拽
                    'dragPrecisionY': 1 # Y轴拖拽精度（整数）
                },
            },
        },
        'series': [
            {'name': 'A', 'data': [[20, 10], [30, 20], [40, 30]]},
            {'name': 'B', 'data': [[50, 40], [60, 50], [70, 60]]},
        ],
    },
    extras=['draggable-points'], # 启用可拖拽点依赖
    on_point_click=lambda e: ui.notify(f'点击: {e}'), # 点击事件
    on_point_drag_start=lambda e: ui.notify(f'拖拽开始: {e}'), # 拖拽开始事件
    on_point_drop=lambda e: ui.notify(f'释放: {e}') # 拖拽释放事件
).classes('w-full h-64')
ui.run()
```

4. 图表动态更新

修改图表的 `options` 属性后，调用 `chart.update()` 方法即可刷新客户端图表，支持数据、样式、配置的实时变更（如示例 1 中的数据更新功能）。

核心逻辑：

- 1. 直接修改 `chart.options` 中的配置（如数据、坐标轴、样式）；
- 2. 调用 `chart.update()` 触发客户端图表重新渲染。

四、组件属性

`ui.highchart` 继承自 NiceGUI 基础 `Element` 类，支持以下常用属性：

属性名	类型	说明	
<code>classes</code>	<code>str</code>	元素的 CSS 类（如 <code>w-full h-64</code> 控制宽高）	

属性名	类型	说明	
<code>client</code>	<code>Client</code>	组件所属的客户端实例	
<code>html_id</code>	<code>str</code>	HTML DOM 中的元素 ID (2.16.0 + 版本支持)	
<code>is_deleted</code>	<code>bool</code>	组件是否已被删除	
<code>is_ignoring_events</code>	<code>bool</code>	组件是否忽略事件	
<code>parent_slot</code>	<code>`Slot`</code>	<code>None`</code>	组件的父插槽
<code>props</code>	<code>Props[Self]</code>	组件的属性 (适配 Vue/Quasar 特性)	
<code>style</code>	<code>Style[Self]</code>	组件的内联样式	
<code>visible</code>	<code>BindableProperty</code>	组件可见性 (支持双向绑定)	

五、核心方法

ui.highchart 支持 NiceGUI 基础组件的所有方法，以下是常用核心方法：

方法名	参数	说明
<code>update()</code>	-	刷新图表，修改 <code>options</code> 后必须调用
<code>set_visibility(visible: bool)</code>	<code>visible</code> : 是否可见	控制图表显示 / 隐藏
<code>tooltip(text: str)</code>	<code>text</code> : 提示文本	为图表添加鼠标悬浮提示
<code>delete()</code>	-	删除图表及所有子元素
<code>bind_visibility(target_object, ...)</code>	目标对象、属性名等	绑定图表可见性到其他对象属性 (双向绑定)
<code>on(type: str, handler: Callable)</code>	<code>type</code> : 事件类型; <code>handler</code> : 回调	监听自定义事件 (如 "click")
<code>run_method(name: str, *args)</code>	<code>name</code> : 方法名; <code>args</code> : 参数	调用客户端 Highcharts 原生方法

常用方法示例

```
# 1. 绑定可见性到变量
from nicegui import ui

show_chart = True
chart = ui.highchart({'chart': {'type': 'line'}, 'series': [{'data': [1,2,3]}]})
chart.bind_visibility(show_chart, 'value') # 当 show_chart 变化时，图表可见性同步变更

# 2. 添加悬浮提示
chart.tooltip('这是一个折线图')

# 3. 监听自定义事件
chart.on('mousedown', lambda e: ui.notify('鼠标按下'))
```

六、注意事项

- 许可限制**：Highcharts 非开源免费，商用需遵守其许可协议，ui.highchart 仅提供集成能力，许可责任由开发者承担。
- 依赖兼容性**：`extras` 参数指定的依赖名称需与 Highcharts 官方模块名一致（如 `solid-gauge`、`draggable-points`），否则会导致模块加载失败。
- 性能优化**：频繁更新图表时，建议批量修改 `options` 后再调用 `update()`，避免多次刷新导致性能问题。
- 事件参数**：`on_point_click` 等回调的 `e` 参数包含 Highcharts 原生事件数据（如点坐标、数据值），可通过 `e.args` 访问详细信息。

七、扩展场景

- 股票图**：设置 `type="stockChart"` 并配置股票数据，实现 K 线图展示；
- 地图**：通过 `type="mapChart"` 结合地图数据和 `extras=['map']`，实现地理数据可视化；
- 复合图表**：在 `series` 中配置多种类型（如折线 + 柱状），实现多维度数据对比；
- 实时数据监控**：结合异步任务（如 `asyncio`），定时更新 `options` 并调用 `update()`，实现实时数据可视化。

通过以上特性，ui.highchart 可满足从简单数据展示到复杂交互式可视化的各类需求，且与 NiceGUI 生态深度融合，便于快速构建完整的 Web 应用。